

ATM SIMULATOR – PYTHON PROJECT

OVERVIEW OF THE PROJECT

The ATM Simulator is a lightweight, menu-driven banking application developed in Python. It simulates basic ATM operations through a command-line interface, allowing a user to securely access an account using a PIN and perform common banking transactions.

The application provides four main operations: checking the account balance, depositing money, withdrawing money, and exiting the ATM. The program starts with an initial balance of **₹10,000** and updates the balance whenever a successful deposit or withdrawal is performed.

The project demonstrates fundamental Python programming concepts including functions, conditional statements, loops, user input, type conversion, arithmetic operations, return values, and input validation.

FEATURES

PIN Authentication

- Requires the user to enter a PIN before accessing the ATM menu.
- Correct PIN allows access to ATM operations.
- Incorrect PIN cancels the transaction.

Balance Checking

- Displays the user's current account balance.

Money Deposit

- Accepts a deposit amount.
- Checks whether the amount is greater than zero.
- Updates and displays the new balance.

Money Withdrawal

- Accepts the withdrawal amount.
- Checks whether the amount is positive.
- Checks whether sufficient balance is available.
- Deducts the amount and displays the remaining balance.

ATM Menu

- Check Balance
- Deposit Money
- Withdraw Money
- Exit

Input Validation & Error Handling

- Incorrect PIN.
- Invalid menu choices.
- Zero or negative transaction amounts.
- Withdrawal amounts greater than the available balance.

TECHNICAL SPECIFICATIONS

Language: Python 3

Application Type: Command-Line Interface

Program Type: Menu-Driven Application

Primary Functions:

```
check_balance()
deposit_money(balance)
withdraw_money(balance)
```

Python Concepts Used:

- Functions
- Function parameters
- Return values

- Variables
- input()
- Type conversion using float() and int()
- if, elif, else
- while loop
- Comparison operators
- Arithmetic operators
- User input validation

HOW IT WORKS

1. Program Initialization

The program displays an ATM welcome message, defines the ATM PIN, and requests the user to enter a PIN.

2. Authentication

If the entered PIN matches the stored PIN, the program initializes the account balance to ₹10,000 and displays the ATM menu. If the PIN is incorrect, the transaction is cancelled.

3. Checking Balance

When option 1 is selected, `check_balance(balance)` displays the current balance.

4. Depositing Money

When option 2 is selected, `deposit_money(balance)` accepts the amount, validates it, updates the balance, and returns the new balance.

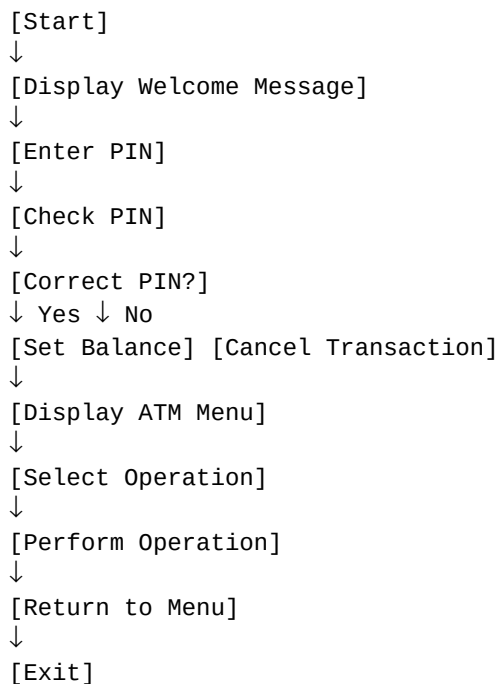
5. Withdrawing Money

When option 3 is selected, `withdraw_money(balance)` validates the amount and available balance. A valid withdrawal reduces the balance and returns the updated value.

6. Exiting the ATM

When option 4 is selected, a thank-you message is displayed and the while loop is terminated.

PROGRAM WORKFLOW



FUNCTION IMPLEMENTATION

`check_balance(balance)`

Displays the current account balance.

```
def check_balance(balance):  
    print("Your current balance is: ■", balance)
```

deposit_money(balance)

Accepts a deposit amount, validates that it is positive, adds it to the balance, and returns the updated balance.

```
balance = balance + amount  
return balance
```

withdraw_money(balance)

Accepts a withdrawal amount and checks whether it is positive and does not exceed the available balance. A successful withdrawal returns the updated balance.

DATA HANDLING

The program uses a numeric variable to maintain the account balance. The initial balance is ■10,000.

```
balance = 10000
```

Deposit:

```
New Balance = Current Balance + Deposit Amount
```

Withdrawal:

```
New Balance = Current Balance - Withdrawal Amount
```

No external database or file storage is used in the current implementation.

DESIGN DECISIONS & RATIONALE

1. Programming Language: Python

Python was selected because its simple syntax is suitable for implementing a beginner-friendly banking simulation.

2. Interface: Command-Line Interface

A command-line interface keeps the application simple and focuses on programming logic.

3. Modular Function Design

Separate functions are used for balance checking, depositing money, and withdrawing money.

4. Menu-Driven Operation

The menu allows users to repeatedly perform ATM operations without restarting the program.

5. Validation Through Conditional Statements

if, elif, and else statements prevent invalid transactions such as negative amounts and withdrawals exceeding the available balance.

INPUT VALIDATION

Deposit Validation

```
if amount > 0:
```

Only positive deposit amounts are accepted.

Withdrawal Validation

```
if amount <= 0:
```

Invalid amount

```
elif amount > balance:
```

Insufficient balance

Menu Validation

If the user enters an option other than 1, 2, 3, or 4, the program displays an invalid-choice message.

SAMPLE EXECUTION

```
=====
WELCOME TO ATM
=====
```

Enter your PIN: 1234

```
----- ATM MENU -----  
1. Check Balance  
2. Deposit Money  
3. Withdraw Money  
4. Exit  
-----
```

Enter your choice: 1
Your current balance is: ■ 10000

Deposit Example

Enter your choice: 2
Enter amount to deposit: ■2000

■ 2000 deposited successfully.
Updated balance: ■ 12000

Withdrawal Example

Enter your choice: 3
Enter amount to withdraw: ■3000

Please collect your cash.
Remaining balance: ■ 9000

Insufficient Balance Example

Enter amount to withdraw: ■15000
Insufficient balance.

TESTING APPROACH

- Correct PIN and incorrect PIN.
- Balance checking.
- Positive, zero, and negative deposit amounts.
- Positive withdrawal within available balance.
- Withdrawal greater than available balance.
- Zero and negative withdrawal amounts.
- Valid and invalid menu selections.
- Exit operation.

CHALLENGES FACED

- Maintaining the updated balance between different functions.
- Passing the balance as a function parameter.
- Returning the updated balance after deposits and withdrawals.
- Preventing withdrawals greater than the available balance.
- Handling invalid transaction amounts.
- Keeping the ATM menu running until Exit is selected.
- Providing clear messages for successful and unsuccessful operations.

OUTPUT

The program produces an ATM welcome message, PIN authentication result, current balance, deposit confirmation, updated balance, withdrawal confirmation, remaining balance, invalid transaction messages, insufficient-balance messages, invalid menu messages, and an exit message.

LEARNING OUTCOMES

- Creating and calling user-defined functions.
- Passing values through function parameters.
- Returning updated values from functions.
- Using if, elif, and else.
- Using a while loop for repeated menu operations.
- Taking and converting user input using `int()` and `float()`.
- Performing arithmetic operations.
- Applying validation rules.
- Building a menu-driven command-line application.

FUTURE ENHANCEMENTS

- Multiple user accounts.
- Secure PIN handling.
- PIN change functionality.
- Mini statement or transaction history.
- Fund transfer between accounts.
- Daily withdrawal limits.
- Account creation.
- File or database-based balance storage.
- Transaction receipts.
- Graphical user interface.
- Improved exception handling for non-numeric input.
- Multiple ATM transaction types.

CONCLUSION

The ATM Simulator demonstrates how Python can be used to create a simple interactive banking application through a command-line interface. The project combines user-defined functions, conditional statements, loops, input handling, arithmetic operations, return values, and validation to simulate common ATM transactions.

The program provides basic ATM functionality including PIN authentication, balance checking, deposits, withdrawals, insufficient-balance checking, and menu-based navigation. Its modular structure provides a foundation for future improvements such as multiple accounts, transaction history, database storage, enhanced security, and a graphical interface.

PROJECT SOURCE CODE

The implementation is contained in the Python source program. It defines separate functions for balance checking, money deposits, and money withdrawals, while the main program controls PIN authentication, menu navigation, balance management, and program termination.

REFERENCES

- [Python Programming Language Documentation.](#)
- [Python Built-in Functions Documentation.](#)
- [Python Input and Output Documentation.](#)
- [Python Conditional Statements and Loops Documentation.](#)